

Übungen

Parallele und Reaktive Programmierung in Java

Marwan Abu-Khalil
2020

Inhalt

- **Thread**
 - Thread Parallelität Übungen S. 9
 - Thread Synchronisation und Monitor Übungen S. 16
- Fork-Join-Task Übungen S. 26
- Parallel-Streams Übungen S. 33
- Reactive-Streams Rx-Java Übungen S. 42
- Hardwarenahe Parallelität Übungen S. 50

Thread-Parallelität Übungen

1. Thread Übungsaufgaben

- Para-1: Triviale Parallelisierung mit Threads: Hello-World S. 10
(Thread-API und -Verhalten kennenlernen)
- Para-2: Fibonacci-Parallelisierung mit Threads S. 11
(Performance durch Parallelisierung rekursiver Algorithmen)
- Para-3: Quicksort-Parallelisierung mit Threads S. 13
(Performance durch Parallelisierung rekursiver Algorithmen)

EinfachMittelSchwer

2. Threadpool, Executors Übungsaufgaben

- Para-4: Threadpool-Performance S. 15
(Thread-Pool und Future API kennenlernen)

Einfach

Thread Synchronisation und Monitor Übungen

1. Übungen zur Synchronisation

- Übung Synch-1: Linked-List inkonsistent S. 17
(Inkonsistenzen erzeugen und beheben)
- Synch-2: Ein unkorrekt synchronisiertes Programm reparieren (Stack) S.18
(Relevanz von synchronized verstehen)
- Synch-4: Phaser S. 21
(Komplexe Synchronisationsabläufe)
- Synch-5: Reader-Writer Lock S. 22
(Mechanik des Expliziten Lockings am Beispiel Reader-Writer erlernen)

Einfach

Einfach

Mittel

Schwer

2. Übungen zum Monitor

- Monitor-2: Hello-World: Abwechslung zweier Threads mit Monitor S. 24
(Monitor-Pattern verstehen)
- Monitor-3: Producer-Consumer mit selbst geschriebener Monitor-Queue S. 25
(Monitor auf reales Problem anwenden)

Mittel

Schwer

Fork-Join-Tasks Übungen

1. Task API und Performance

- Task-1: Schleife parallelisieren S. 27
- Task-2: Baum aus Tasks bauen
- Task-3: Fibonacci Tasks-Parallelisierung S. 28
- Task-4: Quicksort Tasks-Parallelisierung S. 30

Einfach

Mittel

Mittel

Mittel

2. Task Verhalten

- Task-7: Blocking Pool S. 31
- Task-9: Steal-Count S. 32

Einfach

Mittel

Parallel-Streams Übungen

Übungsthema 1: Parallele Map-Reduce Pipelines

- [PS 1.1 Stream API kennenlernen S. 34](#)
Map-Reduce und Parallelität an einem einfachen Beispiel
- [PS 1.2 Parallel Map-Reduce Performance S. 35](#)
Erkennen, dass Parallelität die Stream-Performance massiv verbessert
- [PS 1.3 API der Methode flatMap\(\) S. 36](#)
Verschachtelte Datenstrukturen mit der Stream-API verarbeiten
- [PS 1.4 Spliterator-Performance \(LinkedList vs. ArrayList\) S. 37](#)
Zeigen dass Spliteration einer Array-List schneller ist die einer Linked-List

EinfachMittelSchwerMittel

Übungsthema 2: Spliterator API

- [PS 2.1 Spliterator Implementierung und Verhalten S. 38](#)
Eigenen grundlegenden Spliterator schreiben

Mittel

Übungsthema 3: Laufzeitverhalten

- [PS 3.1 Threshold Array-Spliterator S. 39](#)
Spliterator schreiben, der den Threshold manipuliert

Mittel

Reactive-Streams RxJava Übungen

Übungsthema 1: RxJava API Grundlagen

- [RX 1.1: API Basic Parallelism S. 43](#)
RxJava API Grundlagen in Bezug auf Parallelität und Nebenläufigkeit
- [RX 1.3 Flowable vs. Observable, Back-Pressure S. 45](#)
Back-Pressure in Flowables verstehen
- [RX 1.4 API für Verzweigen und Zusammenführen S. 46](#)
flatMap(), groupBy() verstehen, als Basis für Parallelität

Einfach**Mittel****Mittel**

Übungsthema 2: Concurrency in RxJava

- [RX 2.1 Concurrency-Performance S. 47](#)
Asynchroner Stream und dessen Performance-Vorteil

Mittel

Übungsthema 3: Parallelität in RxJava

- [RX 3.1 Parallel Performance mit flatMap\(\) und groupBy\(\) S. 49](#)
Parallelisierung von Streams verstehen, Performance-Potential erkennen

Schwer

Hardware-nahe Parallelität Übungen

- Atomic-1: Atomares i++ auf Basis von CAS S. 51
- Atomic-2: Spinlock mit CAS programmieren S. 52
- Memory-Model-1: volatile: Sichtbarkeit der Variablenzuweisung S. 53
- Memory-Model2: Peterson's Lock: Reorderings und Memory-Barriers S. 54

EinfachMittelMittelSchwer

Thread Parallelität Übungen

1. Thread Übungsaufgaben

- Para-1: Triviale Parallelisierung mit Threads: Hello-World S. 10
(Thread-API und -Verhalten kennenlernen)
- Para-2: Fibonacci-Parallelisierung mit Threads S. 11
(Performance durch Parallelisierung rekursiver Algorithmen)
- Para-3: Quicksort-Parallelisierung mit Threads S. 13
(Performance durch Parallelisierung rekursiver Algorithmen)

EinfachMittelSchwer

2. Threadpool, Executors Übungsaufgaben

- Para-4: Threadpool-Performance S. 15
(Thread-Pool und Future API kennenlernen)

Einfach

Übung Para-1 Triviale Parallelisierung mit Threads: „Hello World“

Schreiben Sie ein Programm mit zwei Threads. Ein Thread schreibt in einer unendlichen Schleife das Wort „Hello“ auf die Konsole, der andere fortwährend das Wort „World“.

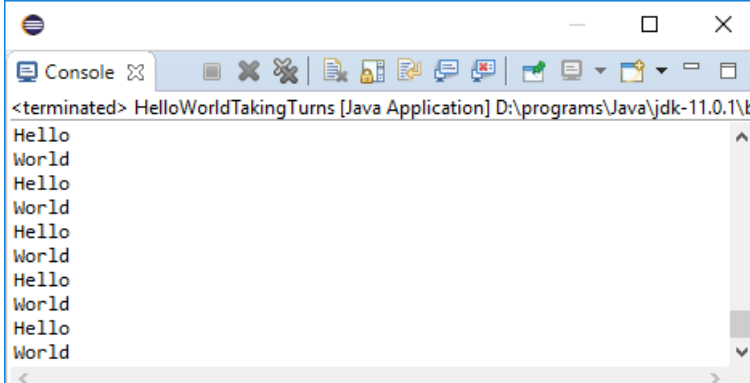
- Die Threads schreiben in beliebiger Abfolge
- Verfeinern Sie die Lösung von a) so, dass die Worte immer genau abwechselnd ausgeschrieben werden.
Erreichen Sie dies, ohne einen Monitor zu verwenden, d.h. ohne `wait()` und `notify()`, und ohne `synchronized`

Lernziel:

Thread API kennenlernen, beobachten, dass das OS die Threads abwechselnd bzw. gleichzeitig schedult.

Musterlösung

seminar\exercises\thread\parallel\hello_world_threads\



```
<terminated> HelloWorldTakingTurns [Java Application] D:\programs\Java\jdk-11.0.1\b
Hello
World
Hello
World
Hello
World
Hello
World
Hello
World
```

Übung Para-2

Fibonacci-Algorithmus Parallisierung mit Threads

Parallelisieren Sie den Fibonacci-Algorithmus mit Hilfe von Threads, messen Sie den Speedup

Der sequentielle Algorithmus steht in den Vortragsfolien und ist in den Musterlösungen implementiert.

Finden Sie eine Lösung, deren Laufzeit wesentlich kürzer ist, als die des sequentiellen Algorithmus.
Ideal wäre: $\text{Parallele Laufzeit} = \text{Sequentielle Laufzeit} / \text{Anzahl CPUs}$

Hinweise:

- Beginnen Sie mit einer naiven Parallelisierung, die für jede Teilaufgabe einen Thread startet
- Begrenzen Sie in einem zweiten Schritt die Anzahl der Threads

Lernziel:

- Erkennen, dass ein massiver Speedup möglich ist
- Erkennen, dass eine große Threadanzahl Probleme verursacht
- Diese Probleme lösen können

Musterlösung:

`exercises/thread/parallel/fibonacci_threads/`

Para-2: Hinweise zu Fibonacci mit Threads

Algorithmische Voraussetzung

```
class Fibo{
    long arg;
    long result;

    Fibo (long arg){
        this.arg = arg;
    }

    public void calculate(){
        if(arg <= 1){
            result = 1;
        }else{
            // Start new Threads for recursion
            Fibo fibo_n_1 = new Fibo (arg - 1);
            Fibo fibo_n_2 = new Fibo (arg - 2);

            fibo_n_1.calculate();
            fibo_n_2.calculate();
            // use threads results
            result = fibo_n_1.result + fibo_n_2.result;
        }
    }
}
```

Hier Parallelisieren

Hier Parallelisieren

Übung Para-3: Parallelisierung des Quicksort Algorithmus mit Threads

Parallelisieren Sie den Quicksort-Algorithmus mit Hilfe von Threads und messen Sie den Speedup.

Der sequentielle Algorithmus steht auf der folgenden Folie, und er ist in den Musterlösungen implementiert.

Finden Sie eine Lösung, deren Laufzeit wesentlich besser ist als die des sequentiellen Algorithmus. Ideal wäre: Laufzeit = Ursprüngliche Laufzeit / Anzahl CPUs

Hinweise:

- Begrenzen Sie die Anzahl der Threads, die gestartet werden.
- Benutzen Sie jedoch keinen Threadpool o.ä.

Lernziel:

- Erkennen, dass ein massiver Speedup möglich ist
- Probleme erkennen und lösen, die durch eine zu große Anzahl von Threads entstehen

Musterlösung: seminar\exercises\thread\parallel\quicksort_threads\

Ausgangsbasis zu Übungsaufgabe Para-3: Quicksort sequentiell

```
class QuicksortSequential {  
    // Array of integers to be sorted  
    private int[] data;  
    public QuicksortSequential(int[] data){  
        this.data = data;  
    }  
    public void sort(){  
        sortRecursively(0, data.Length - 1);  
    }  
    void Swap(int i, int j){  
        int tmp = data[i];  
        data[i] = data[j];  
        data[j] = tmp;  
    }  
}
```

```
// Fortsetzung von QuicksortSequential  
void sortRecursively(int l, int u){  
    if (l >= u){  
        return;  
    }  
    int pivotIdx = l;  
    // running indices:  
    int i = l;  
    int j = u + 1;  
    while (true){  
        do{  
            i++;  
        } while (i <= u && data[i]  
            <= data[pivotIdx]);  
        do {  
            j--;  
        } while (data[j] > data[pivotIdx]);  
        if (i > j) break;  
        Swap(i, j);  
    }  
    Swap(pivotIdx, j);  
    // Recursive Calls  
    sortRecursively(l, j - 1);  
    sortRecursively(j + 1, u);  
}
```

Übung Para-4: Threadpool-Performance

- Schreiben Sie eine for-Schleife, die in jedem Schleifendurchlauf eine globale Variable inkrementiert
- Parallelisieren Sie diese Schleife mit Hilfe eines Thread-Pools
- Zerlegen Sie dafür den Wertebereich des Schleifenindex in mehrere Teilbereiche, und berechnen Sie das Resultat jedes dieser Teilbereiche mit einem Callable im Thread-Pool
- Nutzen Sie Future-Instanzen, um die Resultate der Teilbereiche zusammenzufügen
- Zeigen Sie, dass Ihre Implementierung schneller ist, als die sequentielle Schleife

Ausgangsbasis: Lange for-Schleife

```
void sequentialLoop() {  
    long counter = 0;  
    for(long i = 0; i < 40000000000L; ++i) {  
        counter++;  
    }  
}
```

- **Musterlösung:**
seminar\exercises\thread\parallel\threadpool_performance

Thread Synchronisation und Monitor Übungen

1. Übungen zur Synchronisation

- Übung Synch-1: Linked-List inkonsistent S. 17
(Inkonsistenzen erzeugen und beheben)
- Synch-2: Ein unkorrekt synchronisiertes Programm reparieren (Stack) S.18
(Relevanz von synchronized verstehen)
- Synch-4: Phaser S. 21
(Komplexe Synchronisationsabläufe)
- Synch-5: Reader-Writer Lock S. 22
(Mechanik des Expliziten Lockings am Beispiel Reader-Writer erlernen)

Einfach

Einfach

Mittel

Schwer

2. Übungen zum Monitor

- Monitor-2: Hello-World: Abwechslung zweier Threads mit Monitor S. 24
(Monitor-Pattern verstehen)
- Monitor-3: Producer-Consumer mit selbst geschriebener Monitor-Queue S. 25
(Monitor auf reales Problem anwenden)

Mittel

Schwer

Übung Synch-1: Linked-List inkonsistent

- a) Zeigen Sie, dass eine Linked-List aus dem JDK inkonsistent wird, wenn mehrere Threads simultan Daten einfügen.
- b) Reparieren Sie das fehlerhafte Verhalten.

Hinweis:

Um die Inkonsistenz nachzuweisen, können Sie z.B. die Linked-List mit `toString()` auf die Shell schreiben.

Lernziel: Erkennen, dass Inkonsistenzen leicht entstehen und fatale folgen haben können

Musterlösung: `seminar\exercises\thread\synchronization\linked_list_inconsistent`

Übung Synch-2: Ein unkorrekt synchronisiertes Programm reparieren (Stack)

Synchronisieren Sie die Stack-Implementierung aus dem Seminar korrekt.

Hinweis:

In Ausgangsbasis zur Musterlösung finden Sie die Implementierung des Stacks, und ein Testprogramm, das fehlerhafte Zustände anzeigt (durch Exceptions). Ihre Synchronisation sollte dazu führen, dass keine Exceptions mehr auftreten.

Lernziel:

- Risiken der unkorrekten Synchronisation verstehen
- Gegenmaßnahmen finden.

Musterlösung:

seminar\exercises\thread\synchronization\stack_unsynchronized

```
public class Stack {  
  
    // Array used for stack-data  
    Object[] stack = new Object[100];  
  
    // Points to top element  
    int stackPointer = -1;  
  
    void push(Object newData){  
        // Insert new data  
        stack[stackPointer + 1] = newData;  
  
        // Increment stack pointer  
        stackPointer++;  
    }  
}
```

Übung Synch-3: List-Writer

Finden Sie den Denkfehler im Programm List-Writer

```
class ListWriter{

    static LinkedList<String> list =
        new LinkedList<String>();

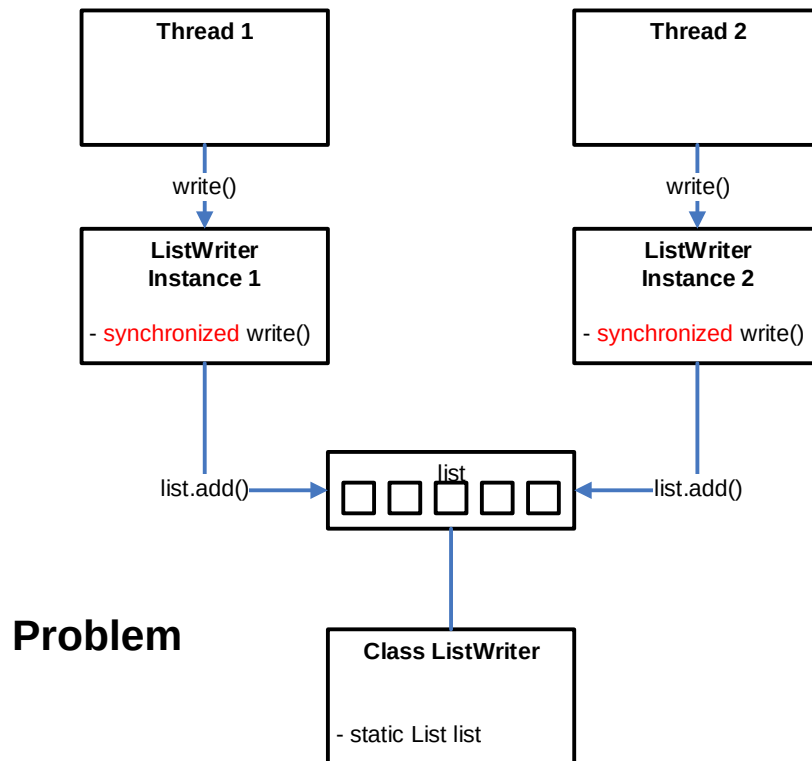
    synchronized void write(){
        for(int i = 0; i < 100; ++i) {
            list.add(String.valueOf(i));
        }
    }

    // call
    new Thread(() -> {
        new ListWriter().write();
    }).start();
    new Thread(() -> {
        new ListWriter().write();
    }).start();
}
```

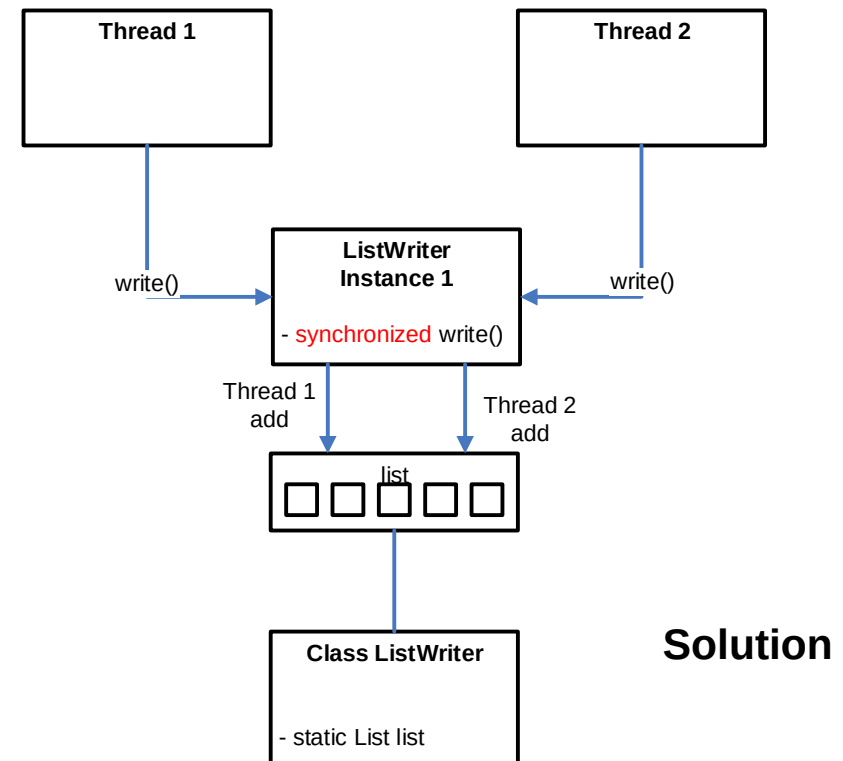
Musterlösung: seminar\exercises\thread\synchronization\list_writer

Lösung zu Übung Synch-3: List-Writer

- Problem: Liste statisches Feld von ListWriter
- Die write() Methode ist eine Instanz Methode
- Die Threads verwenden verschiedene Instanzen
- write() Methoden laufen gleichzeitig



- Lösung:
- Threads verwenden dieselbe ListWriter Instanz
- write() Calls dadurch nacheinander ausgeführt



Übung Synch-4: Phaser

- **Programmieren Sie folgendes Szenario, das einen Wetlauf simuliert:**

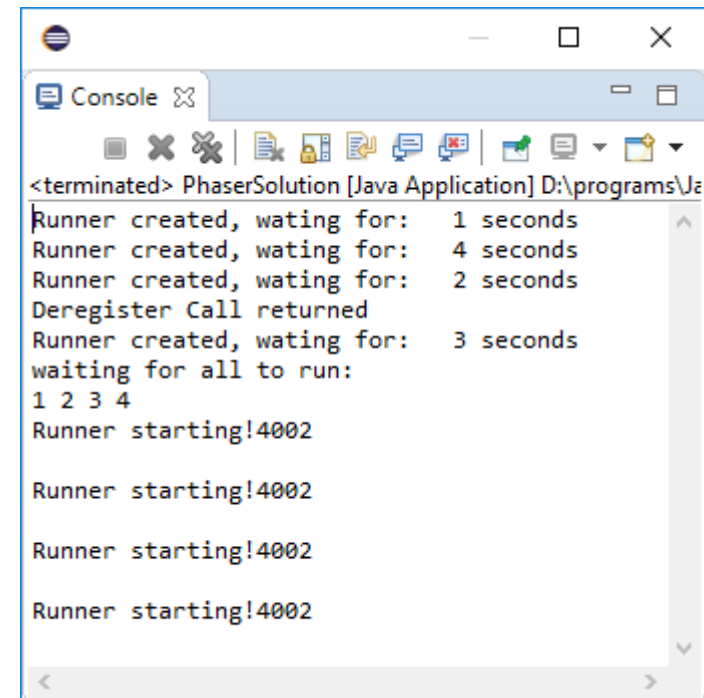
- Mehrere „Läufer“-Instanzen werden erzeugt
- Die Läufer begeben sich zum Start
- Die Läufer brauchen unterschiedlich lange, bis sie am Start ankommen
- In dem Moment, in dem alle Läufer den Start erreicht haben laufen sie alle gleichzeitig los

- **Hinweis: Nutzen Sie folgende APIs**

- **Phaser:** Diese Klasse ermöglicht gleichzeitiges Loslaufen
- **Phaser.register():** Ein Läufer meldet sich an
- **Phaser.arriveAndAwaitAdvance():**
Das ist der zentrale Aufruf: Ankommen an der Schranke und in dem Moment loslaufen, wo auch alle anderen dort ankommen
- **Phaser.deregeister():** Sich am Phaser abmelden, das kann z.B. ein Schiedsrichter tun, der das alles organisiert

- **Musterlösung:**

seminar\exercises\thread\synchronization\phaser



```
<terminated> PhaserSolution [Java Application] D:\programs\Ja
Runner created, waiting for: 1 seconds
Runner created, waiting for: 4 seconds
Runner created, waiting for: 2 seconds
Deregister Call returned
Runner created, waiting for: 3 seconds
waiting for all to run:
1 2 3 4
Runner starting!4002

Runner starting!4002

Runner starting!4002

Runner starting!4002
```

Übung Synch-5: Reader-Writer Lock

- Schreiben Sie eine Klasse, die den Zugriff auf eine Datenstruktur (z.B. eine LinkedList) so schützt, dass zwar viele Threads gleichzeitig aus ihr lesen können, aber nur jeweils ein Thread zu jeder Zeit in diese Struktur schreiben kann.
- Nutzen Sie für die Synchronisation die Klasse `util.concurrent.ReentrantReadWriteLock`
- Durch diese Art des Lockings lässt sich ein Performance-Vorteil erzielen, gegenüber einem Synchronisationsmodell, bei dem zu jedem Zeitpunkt nur ein Thread die Datenstruktur benutzen darf. Weisen Sie diesen Performance-Vorteil nach, indem Sie die Lese- und Schreib-Operationen künstlich verlangsamen (z.B. indem Sie ein Sleep oder eine CPU-intensive Schleife innerhalb der Critical Section einbauen).

Musterlösung: `seminar\exercises\thread\synchronization\rw_lock_list`

Übung Monitor-1: Producer-Consumer mit blockierender Queue aus `java.util.concurrent`

- Schreiben Sie ein Programm mit folgendem Verhalten
 - Ein Producer-Thread legt fortwährend neue Objekte in eine Queue
 - Ein Consumer-Thread entnimmt fortwährend diese Objekte aus der Queue

- **Hinweis:**
 - Verwenden Sie eine blockierende Queue aus `java.util.concurrent`

- **Beobachten Sie folgendes Verhalten**
 - Consumer und Producer laufen gleichzeitig
 - Die Länge der Queue ist begrenzt
 - Wenn der Producer langsamer wird als der Consumer, wartet der Consumer bei leerer Queue

Lernziel

Anwendung einer Monitor-Queue verstehen

Musterlösung: `seminar\exercises\thread\monitor\library_queue`

Übung Monitor-2: Hello-World mit zwei Threads immer genau im Wechsel

Schreiben Sie ein Programm mit zwei Threads. Ein Thread schreibt fortwährend das Wort „Hello“ auf die Konsole, der andere fortwährend das Wort „World“.

Stellen Sie folgendes sicher:

- Die Wörter werden immer genau abwechselnd geschrieben.
- Kein Busy-Waiting! (Wenn ein Thread nicht schreibt verbraucht er auch keine CPU Zyklen)

Lernziele:

- Monitor-Benutzung verstehen
- Unterschied zwischen Busy-Waiting und Blocking verstehen.

Musterlösung: seminar\exercises\thread\monitor\hello_world_monitor

Übung Monitor-3: Producer-Consumer mit selbst geschriebener Monitor-Queue

1. Konzipieren Sie ein „Producer-Consumer Programm“ (zunächst auf Papier, dann im Code):

- Ein Thread (Producer) füllt „Items“, Objekte oder Zahlen, in eine Queue. Wenn die Queue voll ist (100 Elemente enthält), so blockiert dieser Thread. Der Producer wacht wieder auf, sobald Platz in der Queue ist (d.h. sobald die Queue weniger als 100 Elemente enthält).
- Ein anderer Thread (Consumer), entnimmt die Items aus der Queue. Wenn die Queue leer ist, blockiert dieser Thread. Der Consumer wacht wieder auf, sobald es neue Items in der Queue gibt.

2. Programmieren Sie das Design

- Ohne Busy-Waiting und ohne Verwendung blockierender Queues aus `util.concurrent`

3. Verfeinern Sie Ihr Programm:

- Das Programm kann mit beliebig vielen Producern und Consumern umgehen
- Es werden keine unnötigen Weck-Aufrufe getätigt

4. Zeigen sie, dass es bei Verwendung von `notify()` zu Deadlocks kommen kann:

- Z.B. wenn es sehr viele Producer und Consumer gibt, und die Queue kurz ist

5. Optimieren Sie das Programm indem Sie zwei explizite Condition-Variablen verwenden

- Eine für den Zustand „Queue von“ und eine zweite für den Zustand „Queue leer“

Lernziele:

- Producer-Consumer-Implementierung mit Monitor beherrschen
- Deadlock Risiken erkennen und beheben

Musterlösung: `seminar\exercises\thread\monitor\producer_consumer_monitor` und `condition_variable`

Fork-Join-Tasks Übungen

1. Task API und Performance

- Task-1: Schleife parallelisieren S. 27
- Task-2: Baum aus Tasks bauen
- Task-3: Fibonacci Tasks-Parallelisierung S. 28
- Task-4: Quicksort Tasks-Parallelisierung S. 30

Einfach

Mittel

Mittel

Mittel

2. Task Verhalten

- Task-7: Blocking Pool S. 31
- Task-9: Steal-Count S. 32

Einfach

Mittel

Übung Task-1: Schleife parallelisieren

Einfach

- a) Parallelisieren Sie eine langlaufende Schleife mit ForkJoinTasks, und messen Sie den Speedup.
 - b) Entspricht der Speedup-Faktor der Anzahl der Pool-Threads? (Erwartetes Resultat: eher nicht, da Hyperthreading mehr Threads ausführt, als tatsächliche CPUs vorhanden sind)
-
- **Lernziel:** Task-API kennenlernen und Speedup durch Parallelität erzielen
 - **Musterlösung:** `src\seminar\exercises\task\loop_parallel`

Übung Task-2: Baum aus Tasks bauen

Mittel

Bauen Sie einen binären Baum aus Tasks rekursiv auf.

- a. **Node-Count:** Zählen Sie die Anzahl der erzeugten Knoten im Baum
 - Addieren Sie dafür rekursiv die Anzahl der Kinder jedes Knotens.
- b. **Speedup:** Errechnen Sie den Speedup, der durch die Parallelisierung erreicht wird:
 - jeder Task führt eine identische CPU-intensive Methode `fc()` aus
 - Berechnen Sie: $\text{Anzahl der Aufrufe von } fc() \cdot \text{Laufzeit von } fc() / \text{Laufzeit des Task-Baums}$
- c. **Hyperthreading:** Vergleichen Sie den Speedup mit der Anzahl der logischen CPUs.
 - Ist der Speedup-Faktor geringer als die Anzahl CPUs? Was wäre eine Erklärung dafür?

Musterlösung: `src\seminar\exercises\task\task_tree`

Übung Task-3: Fibonacci Tasks-Parallelisierung

Mittel

Parallelisieren Sie den rekursiven Fibonacci Algorithmus mit ForkJoinTasks

- Was kann man daran erkennen?
 - Braucht man einen Threshold?
 - Ist die Task-Parallelisierung schneller als die Thread-Parallelisierung?

Lernziel: Parallelisierung rekursiver Algorithmen mit Tasks beherrschen, Speedup-Potential erkennen, Relevanz und Schwierigkeiten der Threshold-Definition verstehen

Musterlösung: src\seminar\exercises\task\fibonacci_tasks

Übung Task-4: Quicksort Tasks-Parallelisierung

Mittel

Parallelisieren Sie den Quicksort Algorithmus mit ForkJoinTasks

- Was kann man daran erkennen?
 - Braucht man einen Threshold?
 - Ist die Task-Parallelisierung schneller als die Thread-Parallelisierung?

Lernziel: Parallelisierung rekursiver Algorithmen mit Tasks beherrschen, Speedup-Potential erkennen, Relevanz und Schwierigkeiten der Threshold-Definition verstehen

Musterlösung: `src\seminar\exercises\task\quicksort_tasks`

Übung Task-7: Blocking Pool

Einfach

Blocking IO und Langläufer blockieren den Fork-Join Task-Scheduler:

Starten Sie im Common-Pool blockierende Tasks oder langlaufende Tasks.
Die Anzahl dieser Tasks soll größer sein, als die Anzahl der Threads im Pool.

Starten Sie dann weitere Tasks im Common-Pool, und zeigen Sie, dass diese nie geschedult werden.

Hinweis:

- ForkJoinPool. getParallelism() liefert (vereinfacht) die Anzahl der Pool-Threads

Lernziel: Erkennen, dass Blocking-IO oder Langläufer dem Fork-Join-Pool Konzept entgegenstehen

Musterlösung: src\seminar\exercises\task\blocked_pool

Übung Task-9: Steal-Count

Mittel

Zeigen Sie, dass es Situationen gibt, in denen Work-Stealing stattfindet.

Programmieren Sie ein Szenario, bei dem Viele Tasks erzeugt werden. Diese Erzeugung soll auf eine Art und Weise stattfinden, die dazu führt, dass der Fork-Join-Pool einen Großteil dieser Tasks mit Work-Stealing behandelt.

Erklären Sie, wann Work-Stealing stattfindet, und wann es nicht stattfindet.

Hinweis: `ForkJoinPool.getStealCount()` liefert die Anzahl der Work-Stealing Vorgänge im Pool

Lernziel: Erkennen, dass Work-Stealing essentiell ist, um Threads effizient auszulasten

Musterlösung: `src\seminar\exercises\task\steal_count`

Parallel-Streams Übungen

Übungsthema 1: Parallele Map-Reduce Pipelines

- [PS 1.1 Stream API kennenlernen S. 34](#)
Map-Reduce und Parallelität an einem einfachen Beispiel
- [PS 1.2 Parallel Map-Reduce Performance S. 35](#)
Erkennen, dass Parallelität die Stream-Performance massiv verbessert
- [PS 1.3 API der Methode flatMap\(\) S. 36](#)
Verschachtelte Datenstrukturen mit der Stream-API verarbeiten
- [PS 1.4 Spliterator-Performance \(LinkedList vs. ArrayList\) S. 37](#)
Zeigen dass Spliteration einer Array-List schneller ist die einer Linked-List

Einfach

Mittel

Schwer

Mittel

Übungsthema 2: Spliterator API

- [PS 2.1 Spliterator Implementierung und Verhalten S. 38](#)
Eigenen grundlegenden Spliterator schreiben

Mittel

Übungsthema 3: Laufzeitverhalten

- [PS 3.1 Threshold Array-Spliterator S. 39](#)
Spliterator schreiben, der den Threshold manipuliert

Mittel

Übung PS 1.1 Stream API kennenlernen: Map-Reduce und Parallel

a) Bauen Sie einen Stream der folgendes leistet

- Strings in Integers umwandeln („1“ -> 1)
- Ungerade Zahlen herausfiltern
- Quadrate der Zahlen im Stream berechnen
- Maximum dieser Quadrate finden

b) Parallelisieren Sie den Stream aus a)

- Zeigen Sie, dass der Stream tatsächlich parallel ausgeführt wird (z.B. durch Shell Ausgaben)
- Mit wie vielen und welchen Threads wird der Stream ausgeführt? (Ohne Debugger)

c) Spezialisierte Stream Typen

- Benutzen Sie einen spezialisierten Stream-Typ, um das Maximum eines Array vom typ int zu finden.

d) Method-Reference

- Übergeben Sie eine Method-Reference an reduce() um das Maximum wie in a) zu berechnen

Lösungshinweise

```
String[] stringArray =  
    {"7", "2", "3", "12"};  
// Stream creation  
Stream<String> stream =  
    Arrays.stream(stringArray)  
  
// map  
stream.map(a -> Integer.parseInt(a))  
// reduce  
stream.reduce((a,b) -> Math.max(a,b))  
// parallel  
stream.parallel()
```

Musterlösung: [seminar/exercises/stream/mapreduce/u_ps_1_1_map_red_api/](#)

Übung PS 1.2 Parallel Map-Reduce Performance

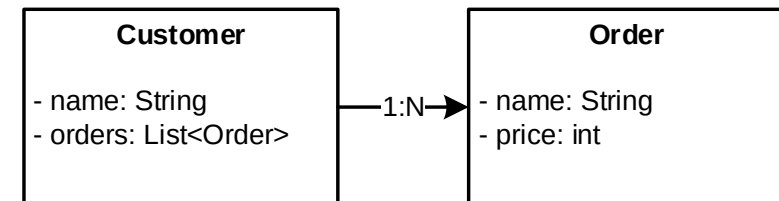
- Zeigen Sie, dass eine einfache Map-Reduce Berechnung durch Parallelität wesentlich beschleunigt werden kann.
- Lösungshinweis:
 - Erzeugen Sie ein großes Array von int-Werten mit Zufallszahlen (ca. 10^8 Einträge)
 - Erzeugen Sie aus dem Array einen Stream
 - filtern Sie alle ungeraden Zahlen aus dem Stream (Stream.filter())
 - Rufen Sie `reduce((a,b) -> max(a,b))` auf dem Stream auf, um das Maximum zu finden.
- Sie sollten durch `parallel()` einen Performance-Vorteil in einem Faktor erreichen, der in etwa der Anzahl der physikalischen CPU-Cores entspricht.
- **Lernziel:** Performance-Vorteil der Parallelisierung eines Streams erkennen
- **Musterlösung:** `seminar\exercises\stream\mapreduce\u_ps_1_2_map_red_performance`

Übung PS 1.3 Stream-API flatMap(): Verschachtelte Datenstrukturen

1. Nutzen Sie die flatMap() Methode, um die Preise aller Orders aller Customers in einem Stream<Customer> aufzusummieren.
 - Customer ist eine verschachtelte Datenstruktur: Customer -> List<Order>
2. Prüfen Sie, ob die Summation mit flatMap() durch Parallelisierung schneller wird, als eine handgeschriebene Schleife, die dieselbe Funktionalität hat.
3. Finden Sie eine Stream-Verarbeitung für diesen Algorithmus, der schneller ist als die handgeschriebene Schleife (Hinweis: Map-Reduce!)

Lösungshinweise:

```
customerStream.  
    flatMap(customer -> customer.orders.stream()).
```



- **Lernziel:** Die verschiedenen API Varianten hinsichtlich Einfachheit und Performance vergleichen
- **Musterlösung:** seminar\exercises\stream\mapreduce\u_ps_1_3_flat_map_api

- **Musterlösung:** seminar\exercises\stream\mapreduce\u_ps_1_4_splitter_performance

Übung PS 2.1 Spliterator Implementierung und Verhalten: Eigenen Spliterator schreiben

- Schreiben Sie einen eigenen Array-Spliterator und benutzen Sie ihn in einer Stream-Verarbeitung. Zeigen Sie, dass die Spliteration in mehreren Threads gleichzeitig erfolgt.
- **Lösungshinweis:** Das Spliterator Interface

```
class SeminarSpliteator implements Spliterator<Integer>{
    @Override
    public boolean tryAdvance(Consumer<? super Integer> action) {
        /* den Consumer auf das aktuelle Element anwenden */
    }
    @Override
    public Spliterator<Integer> trySplit() {
        /* 1. das Array zerteilen.
           2. Einen neuen Spliterator für die linke Hälfte erzeugen,
           4. Den eigenen Range anpassen
           */
    }
}
```

- **Musterlösung:** seminar\exercises\stream\split_collect\u_2_1_spliterator_implementation

Übung PS 3.1 Threshold Array-Spliterator

- Untersuchen Sie, wie groß Leaf-Splits im Standardfall bei einem Array-Stream werden. Finden Sie einen Weg, die Splits größer zu machen.
- **Lösungshinweis:**
 - Array-Spliterator manipulieren, so dass trySplit() null zurückgibt, wenn die gewünschte Größe unterschritten wird.
 - Der Standard Array-Spliterator finden Sie im Code zur Aufgabe

```
public Spliterator<T> trySplit() {  
    // Added for Seminar : THRESHOLD = 1000  
    if((this.fence - this.index) < 1000) {  
        return null;  
    }  
    //...
```

- **Musterlösung:** seminar\exercises\stream\runtimebehavior\u_3_1_threshold_spliterator

Übung PS 3.2 Analyse des parallelen Verhaltens in der Map-Reduce Pipeline: Threads, Splits, Tasks

Schreiben Sie einen einfachen Map-Reduce Stream und untersuchen Sie folgende Fragen

1. Zu Threads und Tasks:

- Wie viele und welche Threads sind in der Split-Phase aktiv (evtl eigenen Spliterator verwenden)?
- Wie viele und welche Threads sind in der Map-Phase aktiv?
- Wie viele und welche Threads sind in der Reduce-Phase aktiv?

2. Zu Splits:

- Wie viele Splits entstehen?
- Wie viele Leaf-Splits entstehen?

3. Zum Threshold

- Wie groß sind die Leaf-Splits?
- Unterhalb welcher Split-Size hört das Framework also auf, weiter zu parallelisieren und geht zu einer sequentiellen Berechnung über?

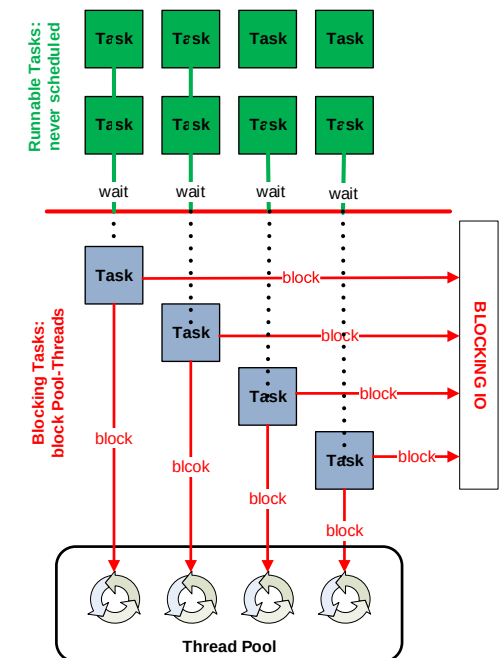
▪ **Musterlösung:** seminar\exercises\stream\runtimebehavior\u_3_2_pipeline_parallelism_analysis

Übung PS 3.4 Blocking IO

- **Schreiben Sie einen Parallel-Stream, der lange, blockierende Aufrufe tätigt, z.B. von der Shell lesen.**
 1. Zeigen Sie, dass der Stream dadurch in einen Deadlock kommt:
Lauffähige Teile des Streams werden nicht abgearbeitet, obwohl keiner der Pool-Threads die CPU benutzt.
 2. Was passiert, wenn Sie einen zweiten Stream starten?
- **Lernergebnis:**
 - BlockingIO führt im Stream zum Deadlock
 - Ist ein Stream blockiert, so sind auch alle anderen Streams blockiert, die den selben ForkJoinPool benutzen.
- **Lösungshinweis:** Eine blockierende IO Operation

```
// Eine blockierende IO Operation
Scanner scanner = new Scanner(System.in);
String input = scanner.nextLine();
```

- **Musterlösung:**
seminar\exercises\stream\runtimebehavior\u_3_4_blocking_io



Reactive-Streams RxJava Übungen

Übungsthema 1: RxJava API Grundlagen

- [RX 1.1: API Basic Parallelism S. 43](#)
RxJava API Grundlagen in Bezug auf Parallelität und Nebenläufigkeit
- [RX 1.3 Flowable vs. Observable, Back-Pressure S. 45](#)
Back-Pressure in Flowables verstehen
- [RX 1.4 API für Verzweigen und Zusammenführen S. 46](#)
flatMap(), groupBy() verstehen, als Basis für Parallelität

Einfach**Mittel****Mittel**

Übungsthema 2: Concurrency in RxJava

- [RX 2.1 Concurrency-Performance S. 47](#)
Asynchroner Stream und dessen Performance-Vorteil

Mittel

Übungsthema 3: Parallelität in RxJava

- [RX 3.1 Parallel Performance mit flatMap\(\) und groupBy\(\) S. 49](#)
Parallelisierung von Streams verstehen, Performance-Potential erkennen

Schwer

Übung RX 1.1: API Basic Parallelism

range(), map(), filter(), subscribeOn()

- **a)** Programmieren Sie auf Basis von RxJava einen endlichen Datenfluss: ein Observable sendet Daten aus, diese werden gefiltert und verarbeitet ein Observer / Subscriber empfängt die Daten.
 - **Lösungsbeispiel:**

```
Flowable.range(0, 1000)
    .map(i -> i/2)
    .filter (i -> (i%2) == 0)
    .subscribe(System.out::println);
```
- **b)** Zeigen Sie, dass die gesamte Pipeline im Main-Thread abläuft. Schreiben Sie dafür in jeder Stufe den aktuellen Thread auf die Shell.
- **c)** Verändern Sie das Verhalten aus b) durch `subscribeOn(Schedulers.computation())`
- **d)** Hängen Sie einen zweiten Subscriber an die Stream-Source aus b) an, und beobachten Sie ob die beiden Streams parallel oder nacheinander ausgeführt werden.
Was ändert sich, wenn Sie `subscribeOn()` verwenden?
- **Lernziel:** RxJava API Grundlagen in Bezug auf Parallelität und Nebenläufigkeit verstehen
- **Musterlösung:** seminar_reactive_streams\exercises\rx\api\rx_1_1_api_basic_parallelism

Übung RX 1.2: Data Generator Stream-Source

- **a) Generator:** Programmieren Sie auf Basis von `Flowable.generate()` einen unendlichen Datenfluss von Integers. Bauen Sie darauf einen trivialen Stream auf, mit `map (i->i)` und `subscribe(i ->{})`

- **Hinweis zur Lösung:**

```
Flowable<Integer> sourceFlowable =  
Flowable.generate(  
    () -> new AtomicInteger(0),  
    (state, emitter) ->  
        {  
            emitter.onNext(state.incrementAndGet());  
        }  
);
```

- **b) subscribeOn():** Zeigen Sie, dass die Erzeugung der Elemente in demjenigen Thread-Pool stattfindet, den Sie mit `subscribeOn()` spezifizieren.
- **c) observeOn():** Zeigen Sie, dass bei Benutzung von `observeOn()` die Erzeugung der Elemente von dem Rest der Pipeline zeitlich entkoppelt wird.
- **Lernziel:** Generator kennenlernen, Wirkung von `subscribeOn()` auf Stream-Source verstehen
- **Musterlösung:** `seminar_reactive_streams\exercises\rx\api\rx_1_2_data_generator`

Übung RX 1.3 Flowable vs. Observable, Back-Pressure

- Zeigen Sie den Unterschied zwischen Observable und Flowable an folgendem Beispiel:

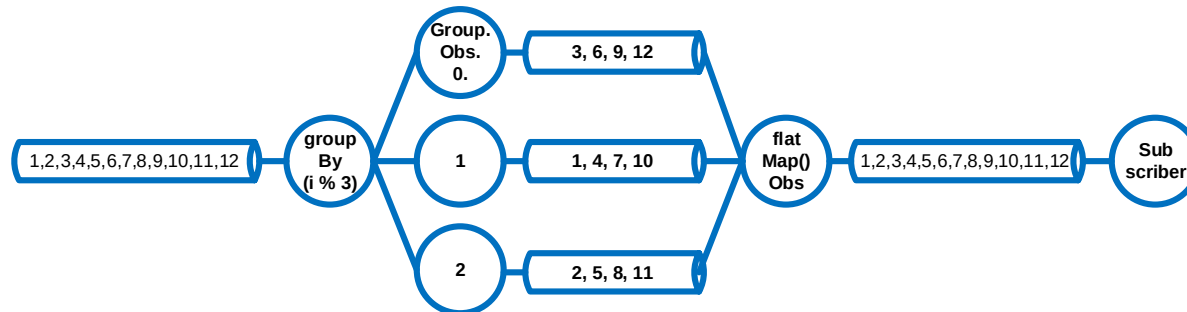
Eine Stream Source erzeugt fortlaufend Integers, z.B. 1 ... 100_000_000.

Der Stream besteht aus folgenden Teilen:

- 1. Ein `map()` - Schritt bearbeitet die Daten.
 - 2. `observeOn()` entkoppelt die Teile des Streams voneinander
 - 3 Ein Subscriber verbraucht die erzeugten Daten. Der Subscriber ist langsamer als der Erzeuger (erreichen Sie das durch `Shell Outputs` oder `Thread.sleep()`)
-
- **a)** Die Stream Source ist ein Flowable. Zeigen Sie, dass Source und Subscriber nur geringfügig auseinander laufen.
 - **b)** Die Stream Source ist ein Observable. Zeigen Sie, dass Source und Subscriber beliebig weit auseinanderlaufen.
 - **Lernziel:** Nutzen von Back-Pressure im Flowable erkennen.
 - **Musterlösung:** `seminar_reactive_streams\exercises\rx\api\rx_1_3_flow_vs_obs`

Übung 1.4 API für Verzweigen und Zusammenführen: flatMap(), groupBy()

- Programmieren Sie einen Stream und gruppieren Sie dessen Daten mit groupBy() zu Gruppen. Verarbeiten Sie jede dieser Gruppen. Fügen Sie die Resultate mit flatMap() wieder zu einem einzigen Stream zusammen.



■ Schrittweise Lösung

- Erzeugen Sie mit `Flowable.range(1,100)` ein Flowable, das 100 Elemente emittiert.
 - Gruppieren Sie diese Elemente mit `groupBy(i -> i% 4)` in vier Gruppen
 - Führen Sie eine triviale map-Operation auf den Elementen jeder Gruppe aus, z.B: `map(i -> i)`
 - Benutzen Sie `flatMap()` um die Gruppen wieder zusammenzuführen
 - Erzeugen Sie Shell-Output, der zeigt, dass jedes Element in genau einer Gruppe verarbeitet wird.
-
- **Lernziel:** Verzweigen und Zusammenführen als Basis für die Parallelisierung von Streams kennen

Übung RX 2.1 Concurrency-Performance

- Erzeugen Sie einen Stream mit zwei Map-Stufen, z.B.
`Observable.range(0,100)`
`.map(i->i)`
`.map(i->i)`
- **a)** Sorgen Sie dafür, dass die beiden Map-Stufen asynchron zueinander ausgeführt werden. Zeigen Sie das durch Shell-Ausgaben, die die jeweiligen Threads ausgeben.
Lösungshinweis: `observeOn(Scheduler.computing())`
- **b)** Fügen Sie in beide Map-Stufen eine langlaufende Funktion ein, und zeigen Sie, dass die asynchrone Version schneller ist, als die sequentielle. Wie viel schneller kann sie werden?
Hinweis: Sie können die langlaufende Funktion `cpuIntensiveCall()` aus der Musterlösung verwenden.
- **Lernziel:** Erkennen, dass ein asynchroner Stream einen Performance-Vorteil bieten kann
- **Musterlösung:**
`seminar_reactive_streams\exercises\rx\concurrency\rx_2_1_concurrency_performance`

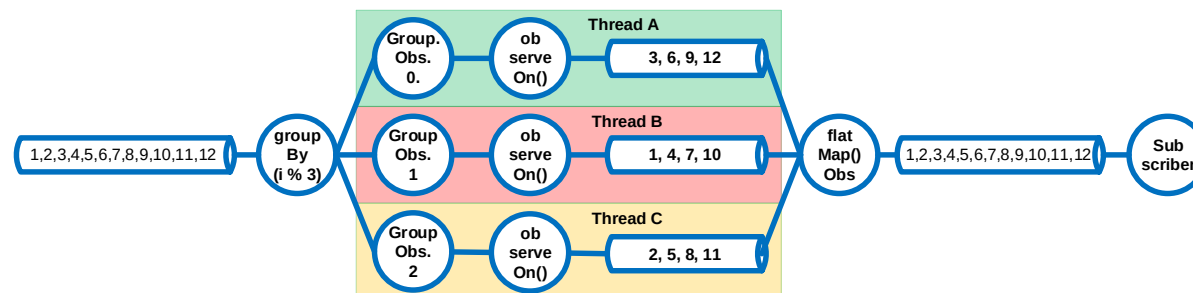
Übung RX 2.2 Concurrency, Memory, Back-Pressure

- Zeigen Sie, dass asynchrone Stufen in einem Stream dazu führen können, dass unbegrenzt viele Elemente in einer Queue gehalten werden. Zeigen Sie ferner, dass dies zum Absturz des Programmes führen kann, wenn diese Elemente viel Memory beanspruchen. Beheben Sie dies durch Back-Pressure.
- **Lösungshinweise**
 - Erzeugen Sie eine Pipeline mit einem Observable als Source:
`Observable.range(1, 100_000_000_0)`
 - Erzeugen Sie in einer ersten Map-Stufe für jedes Element des Streams ein Speicher-Intensives Objekt, das z.B. ein `int[] data = new int[10000]` enthält.
 - Geben Sie der Pipeline eine zweite Map-Stufe, und verlangsamen Sie diese
 - Entkoppeln Sie die beiden Stufen mit `.observeOn(Schedulers.computation())`
 - Zeigen Sie, dass Sie ihr Programm dadurch zum Absturz bringen können (`OutOfMemoryError`)
 - Ersetzen Sie nun das Observable durch ein Flowable und beheben damit das Problem
- **Lernziel:** Erkennen, dass Back-Pressure ein fundamentales Mittel ist, um Risiken hinsichtlich des Ressourcenverbrauches entgegenzuwirken
- **Musterlösung:**
`seminar_reactive_streams\exercises\rx\concurrency\rx_2_2_concurrency_memory_backpressure`

Übung RX 3.1

Parallel Performance mit flatMap() und groupBy()

- Parallelisieren Sie einen Observable-Stream, und zeigen Sie, dass dadurch die Ausführung einer CPU-intensiven Berechnung wesentlich beschleunigt werden kann.
 - **a)** Stream in mehrere unabhängige Sub-Streams zerlegen
 - **b)** Die Sub-Streams parallel ausführen.
 - **c)** CPU-intensive Methode für jedes Stream-Element ausführen, Beschleunigung zeigen
- **Lösungshinweise**
 - groupBy() zerlegt einen Stream in mehrere GroupedObservables
 - flatMap() führt die GroupedObservables wieder zu einem Observable zusammen
 - observeOn() führt jede diese Gruppen in einem eigenen Thread aus



- **Lernziel:** Parallelisierung von Streams verstehen, Performance-Potential erkennen.
- **Musterlösung:**
seminar_reactive_streams\exercises\rx\parallel\rx_1_3_parallel_flatmap_groupby_performance

Hardware-nahe Parallelität Übungen

- Atomic-1: Atomares i++ auf Basis von CAS S. 51
- Atomic-2: Spinlock mit CAS programmieren S. 52
- Memory-Model-1: volatile: Sichtbarkeit der Variablenzuweisung S. 53
- Memory-Model2: Peterson's Lock: Reorderings und Memory-Barriers S. 54

EinfachMittelMittelSchwer

Übung Atomic-1: Atomares i++ auf Basis von CAS

Einfach

Implementieren Sie eine atomare increment Operation auf Basis von AtomicInteger.comapreAndSet()

- Vorgehen
 - Zeigen Sie anhand eines Beispiels, dass eine int Variable, die von zwei Threads simultan inkrementiert wird am Ende einen inkonsistenten Stand hat (z.B. jeder Thread führt 5000 mal i++ durch)
 - Beheben Sie diesen Fehler durch Verwendung von AtomicInteger anstelle des Typs int und durch die Methode compareAndSet() anstelle von des ++ Operators.
- Hinweis
 - AtomicInteger hat eine eigene Increment-Methode, benutzen Sie diese nicht

Lernziel:

- Erkennen, dass der Inkrement-Operator des Datentyps int nicht atomar ist
- Anwendung von Compare-And-Set verstehen

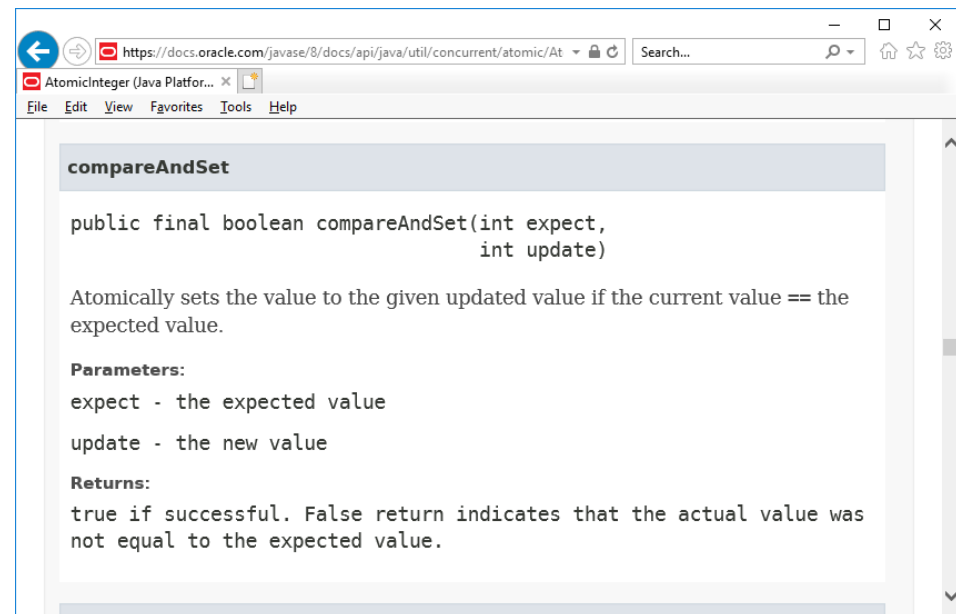
Musterlösung: \seminar\exercises\memorymodel\atomic_increment

Übung Atomic-2: Spinlock mit CAS programmieren

Mittel

- a) Implementieren Sie ein Spinlock auf Basis von CAS (`AtomicInteger.compareAndSet()`).
- b) Zeigen Sie, dass dieses Spinlock funktioniert, z.B. indem Sie damit eine Inkrement-Operation synchronisieren.

- Lösungshinweis: CAS Semantik



Lernziel:

- Funktionsweise von Spinlocks verstehen
- Implementierungsprinzip von Spinlocks verstehen

Musterlösung: `\seminar\exercises\memorymodel\spinlock`

Übung Memory-Model-1

volatile: Sichtbarkeit der Variablenzuweisung

SIEMENS

Mittel

Implementieren Sie zwei Threads, die auf eine gemeinsame Variable zugreifen.

- a) Zeigen Sie, dass es passieren kann, dass ein Thread diese Variable verändert, aber der andre Thread diese Veränderung nicht sieht
- a) Beheben Sie das Verhalten aus a)
 - Lösungshinweis: Wenn Sie zu viele Konsolen-Ausgaben verwenden, verändert sich das Programm-Verhalten. Es kann dann schwierig sein, den gewünschten Effekt zu zeigen.

Lernziel:

- Erkennen, dass die Sichtbarkeit von Variablenzugriffen in einem Multiprozessor eingeschränkt ist

Musterlösung: \seminar\exercises\memorymodel\visibility

Memory-Model-2: Peterson's Lock: Reorderings und Memory-Barriers

SIEMENS

Schwer

- a) Zeigen Sie anhand von Petersons-Lock, dass die CPU Lese- und Schreiboperationen auf Variablen umsortiert.
- b) Beheben Sie dieses Problem durch volatile, aber ohne dass Sie die turn Variable als volatile deklarieren
- c) Erklären Sie das Verhalten
 - Lösungshinweise
 - Nutzen Sie den Petersons-Lock Algorithmus aus der Musterlösung um eine Datenstruktur zu synchronisieren (z.B das Inkrementieren einer int Variable)
 - Zeigen Sie, dass der Peterson-Lock-Algorithmus nicht ohne weiteres funktioniert

Lernziel:

- Erkennen, dass die Umordnungen des Memory-Modells das Programmverhalten verändern können
- Verstehen, dass die Acquire-Release Semantik von volatile dazu führt, dass Memory-Barriers eingefügt werden

Musterlösung: \seminar\exercises\memorymodel\peterson